

Studentu Duomenų Apžvalgos Sistema

3.0

Sugeneruota Doxygen 1.17.0

1 StudentuDuom - studentu duomenų apdorojimo aplikacija	1
1.1 Aprasas	1
1.2 Instaliavimas / paleidimas	1
1.2.1 Reikalavimai	1
1.2.2 Projekto kompiliavimas	1
1.2.3 Programos paleidimas	2
1.3 Versija v3.0	2
1.3.1 Pakeitimai	2
1.4 Testavimas - std::vector vs Vector klase	2
1.4.1 Vidutinis greitis su duomenimis nuo 1000 iki 100mln.	2
1.4.2 Atminties realokacijos	2
1.4.3 - 28 kartus Vector.	3
1.4.4 Testavimo apzvalga	3
1.4.5 Atskiru testu rezultatai	3
1.5 Versija v2.0	4
1.5.1 Pakeitimai	4
1.6 Unit testai	4
1.6.1 Realizuoti testai	5
1.6.2 Testu paleidimas	6
1.6.3 Tikėtinas rezultatas	6
1.7 Versija v1.5	6
1.7.1 Pakeitimai	6
1.7.2 UML klasių hierarchijos diagrama	7
1.7.3 Zmogus - abstrakti klase + rankinis v1.2 pagristas testas	7
1.7.4 v0.2	7
1.7.5 Tyrimas 1	7
1.7.6 v0.4	7
1.7.7 Tyrimas 1	8
1.7.8	8
1.7.9 Tyrimas 2	8
1.7.10	9
1.7.11 v1.0 (subrelease)	9
1.7.12 Tyrimas 1 - Aprasas	9
1.7.13 Aktualus testavimo sistemos parametrai:	9
1.7.14 Tyrimas 1 - Rezultatai	10
1.7.15 v1.0 (optimizuota)	11
1.7.16 Tyrimas 2 - Aprasas	12
2 Hierarchijos Indeksas	12
2.1 Klasių hierarchija	12
3 Klasės Indeksas	12
3.1 Klasės	12

4 Failo Indeksas	12
4.1 Failai	12
5 Klasės Dokumentacija	13
5.1 Studentas Klasė	13
5.1.1 Metodų Dokumentacija	14
5.2 Timer Klasė	14
5.3 Vector< T > Klasė Šablonas	14
5.4 Zmogus Klasė	15
6 Failo Dokumentacija	16
6.1 functions.h	16
6.2 mylib.h	16
6.3 Studentas.h	16
6.4 timer.h	20
6.5 Vector.h	21
Rodyklė	31

1 StudentuDuom - studentu duomenų apdorojimo aplikacija

1.1 Aprasas

Programa skirta studentų duomenų įvedimui, apžvalgai, išvedimui, palyginimui tarpusavy.
Programa naudoja C++20.

Programa kurta VU ISI I kurso studento su tikslu išmokti daugiau apie OOP.

1.2 Instaliavimas / paleidimas

1.2.1 Reikalavimai

- CMake v3.10 arba naujesnė
- C++20 palaikantis kompiliatorius (g++, clang++ arba MSVC)
- Interneto ryšys pirmam paleidimui (GoogleTest parsisiuntimui)

1.2.2 Projekto kompiliavimas

1.2.2.1 1. Sukurti build katalogą

```
mkdir build
cd build
```

1.2.2.2 2. Sugeneruoti projekta

```
cmake ..
```

1.2.2.3 3. Sukompiliuoti projekta

```
cmake --build .
```

1.2.3 Programos paleidimas

1.2.3.1 Linux / macOS

```
./student_program
```

1.2.3.2 Windows

```
.\Debug\student_program.exe
```

1.3 Versija v3.0

1.3.1 Pakeitimai

- Sukurta pilnai veikianti **Vector** klase, paremta (teoriskai) `std::vector`.
 - **Vector** neturi `allocator::traits`, keliu overload'u, taciau `StudentDum` atveju tai yra pilnaverte `std::vector` alternatyva.
 - Kur `std::vector` naudoja `new operator` ar `placement new`, **Vector** naudoja `new T[]`, kad sukurti duomenis. Tai turi trukumu:
 - * **Vector** is karto sukonstruoja visus objektus, kiek gali (capacity), `std::vector` alokuoja `raw` atminti. Tai atveria kelia optimizacijoms - kaip ir minetas `allocator::traits`, kurio **Vector** negali naudoti.
 - Visa `StudentuDum` programa naudoja **Vector**. Is dalies patikrinimui ir general produkto palaikymui.
 - **Vector** ir `std::vector` klases palygintos sparta su iki 100 mln.
 - Rezultatai zemiau bei `Test_Efficiency`
 - Atnaujinta dokumentacija, html, LaTeX, .pdf
 - Atlikti Unit Testai su visais **Vector** metodais. Rezultatai pasiekiami programoje.
-

1.4 Testavimas - `std::vector` vs **Vector** klase

1.4.1 Vidutinis greitis su duomenimis nuo 1000 iki 100mln.

Duom. sk.	<code>std::vector</code> (ms)	Vector (ms)
10,000	0.0000464	0.0000925
100,000	0.0004062	0.0004516
1,000,000	0.0034007	0.0035111
10,000,000	0.0311443	0.0347872
100,000,000	0.3164716	0.3130998

1.4.2 Atminties realokacijos

- Atmintis su 100 mln. duomenu (`<int>`) buvo peralokuota:
 - 47 kartus `std::vector`

1.4.3 - 28 kartus Vector.

1.4.4 Testavimo apzvalga

- `std::vector` bei `Vector` klases yra panasios spartos, taciau `std::vector` yra labiau optimizuota klase. Naudojant `allocator::traits` ir operuojant labiau low-level nei `Vector`, ji greiciau susidoroja su mazesniais duomenu kiekiais.
- `std::vector` ir `Vector` klases naudoja kitokius atminties realokacijos principa, `Vector` alokuoja $2 * capacity$, o `std::vector` dažnai $\sim 1.5 * capacity$. Tai reiskia, kad nors ir `std::vector` peralokavo atminti 19 kartu daugiau - ji ja alokavo labiau efektyviai. Be to, tai parodo, kad atminties realokacija, kopijavimas, move semantika, tvarkant paprastus tipus (kaip `int`), nera labai brangu.

1.4.5 Atskiru testu rezultatai

1.4.5.1 10 000

Nr.	std::vector (ms)	Vector (ms)
1	0.0000864	0.0001092
2	0.0000462	0.0000701
3	0.0000404	0.0001064
4	0.0000384	0.0000611
5	0.0000414	0.0001157
Vid.	0.0000464	0.0000925

1.4.5.2 100 000

Nr.	std::vector (ms)	Vector (ms)
1	0.0006316	0.0006274
2	0.0003711	0.0004005
3	0.0003387	0.0005090
4	0.0003518	0.0003615
5	0.0003377	0.0003597
Vid.	0.0004062	0.0004516

1.4.5.3 1 000 000

Nr.	std::vector (ms)	Vector (ms)
1	0.0034704	0.0040201
2	0.0040731	0.0033310
3	0.0030384	0.0035121
4	0.0031859	0.0032517
5	0.0036356	0.0034405
Vid.	0.0034007	0.0035111

1.4.5.4 10 000 000

Nr.	std::vector (ms)	Vector (ms)
1	0.0291172	0.0352419
2	0.0328546	0.0355669
3	0.0303506	0.0339828
4	0.0318232	0.0349377
5	0.0313760	0.0345071
Vid.	0.0311443	0.0347872

1.4.5.5 100 000 000

Nr.	std::vector (ms)	Vector (ms)
1	0.321536	0.309383
2	0.315756	0.311027
3	0.315996	0.319193
4	0.313160	0.310593
5	0.315910	0.315307
Vid.	0.3164716	0.3130998

1.5 Versija v2.0

1.5.1 Pakeitimai

- Doxygen automatiskai sugeneruota dokumentacija (html bei LaTeX)
 - .pdf failas sugeneruotas pasinaudojus Overleaf
- Realizuoti Unit testai naudojant GoogleTest framework'a
 - Aprasyta naudojimosi instrukcija, pakeisti programos veikimo reikalavimai
- Aprasyti praleisti release'ai siame README.md faile, sukurtas trumpas aprasas

1.6 Unit testai

Projektui realizuoti Unit Testai naudojant **GoogleTest** framework'a.

Testai skirti patikrinti:

- skaiciavimo funkcijas
- studento duomenu nuskaityma
- operatoriu overloads
- Rule of Five realizacija

1.6.1 Realizuoti testai

1.6.1.1 Skaiciavimo testai (CalcTests)

Testas	Tikslas
VidurkisTest	Patikrina <code>calc_vidurkis()</code>

Testas	Tikslas
MedianaOddTest	Patikrina mediana nelyginiu pazymiu kiekiu
MedianaEvenTest	Patikrina mediana su lyginiu pazymiu kiekiu

1.6.1.2 Studento klases testai (StudentasTests)

Testas	Tikslas
ReadStudentTest	Patikrina konstruktoriu su istream (kuris nuskaito duomenis is srauto)
GalutinisVidurkisTest	Patikrina galutini bala su vidurkiu
OutputOperatorTest	Patikrina operator<<

1.6.1.3 Rule of Five testai (RuleOfFiveTests)

Testas	Tikslas
CopyConstructorTest	Patikrina Copy C-tor
MoveConstructorTest	Patikrina Move C-tor
CopyAssignmentTest	Patikrina Copy Assignment operator
MoveAssignmentTest	Patikrina Move Assignment operator

1.6.2 Testu paleidimas

1.6.2.1 Paleisti visus testus

Windows:

```
.\tests\Debug\run_tests.exe
```

arba:

```
ctest
```

Linux / macOS:

```
./tests/run_tests
```

arba:

```
ctest
```

1.6.3 Tiketinas rezultatas

```
[=====] Running 10 tests from 3 test suites.
[-----] 3 tests from CalcTests
[-----] 3 tests from StudentasTests
[-----] 4 tests from RuleOfFiveTests

[ PASSED ] 10 tests.
```

1.7 Versija v1.5

1.7.1 Pakeitimai

- Klase [Studentas](#) pakeista i `Studentas::Zmogus`, prideta abstrakti klase [Zmogus](#).

– Klasiu hierarchija aprasyta UML diagramoje zemiau.

- Atliktas rankinis testavimas, vos neidentiskas i v1.2, su skirtumu, kad iliustruota, jog [Zmogus](#) yra abstrakti klase.

– Demonstracija pateikta nuotraukoje zemiau.

- v1.2 logika ir veikimo principai islike.

1.7.2 UML klasiu hierarchijos diagrama

1.7.3 Zmogus - abstrakti klase + rankinis v1.2 pagristas testas

 <hr> @section autotoc_md51 Versija v1.2 @subsection autotoc_md52 Pakeitimai - Sukurti Rule of Five metodai Studentas klasei - Copy Assignment, Move Assignment, Copy C-tor, Move C-tor, Destruktorius - Destruktorius paliktas kaip <tt>default</tt>, del to kad visoje programoje naudojami STL konteineriai, taciau ateityje gali reiketi - Sukurti I/O operatoriu overload'ai Studentas klasei - Refactor'intos kai kurios funkcijos, kad naudotu siuos pakeitimus <hr> @section autotoc_md54 Versija v1.1 @subsection autotoc_md55 Pakeitimai - Klonuota repozitorija senosios pagrindu - Pereita nuo Studentas struct i Studentas class <hr> @section autotoc_md57 Testavimas - Testavimo rezultatu nuotraukos yra ikeltos Assets aplanke

1.7.4 v0.2

Tyrimas buvo atliktas su Visual Studio 2022 /O2 optimizacijos nustatymais.

Tyrimo 1 failai buvo istrinti prieš kiekviena bandyma.

Testavimo metu testavimo sistemoje nebuvo ijungtos jokios kitos programos.

1.7.5 Tyrimas 1

1.7.5.1 1. Anksciau sugeneruotu failu nuskaitymas:

	1000	100000	1000000
1.	0.0478s	0.619s	2.965s
2.	0.0489s	0.616s	2.894s
3.	0.0478s	0.647s	2.983s
4.	0.0479s	0.593s	2.979s
5.	0.0498s	0.640s	2.966s
Vid.	0.04844s	0.623s	2.9574s

1.7.6 v0.4

Tyrimai buvo atlikti su Visual Studio 2022 /O2 optimizacijos nustatymais.

Tyrimo 1 ir tyrimo 2 failai buvo istrinti prieš kiekviena bandyma, isskyrus tyrimo 2 failus, su kuriais buvo testuojamas skaitymo greitis (Tyrimas 2.1).

Testavimo metu testavimo sistemoje nebuvo ijungtos jokios kitos programos.

1.7.7 Tyrimas 1

1.7.7.1 1. Failu generavimas ir uždarymas:

	1000	10000	100000	1000000	10000000
1.	0.003s	0.03s	0.31s	3.29s	31.4s
2.	0.004s	0.03s	0.34s	3.36s	31.9s
3.	0.004s	0.03s	0.34s	3.3s	31.5s
4.	0.003s	0.04s	0.35s	3.32s	31.7s
Vid.	0.0035s	0.0325s	0.335s	3.318s	31.625s

1.7.7.2 Tyrimo 1 output ekrane:

1.7.8 ``

Kitas nuotraukas galima rasti repozitorijos assets aplanke (v0.4/Assets/...).

1.7.9 Tyrimas 2

1.7.9.1 1. Duomenų nuskaitymas iš anksciau sugeneruotu failu:

	1000	10000	100000	1000000	10000000
1.	0.005s	0.04s	0.46s	4.62s	47.37s
2.	0.007s	0.04s	0.47s	4.78s	47.66s
3.	0.005s	0.05s	0.47s	4.75s	47.72s
4.	0.006s	0.05s	0.47s	4.7s	47.46s
Vid.	0.00575s	0.045s	0.4675s	4.7125s	47.5525s

1.7.9.2 2. Studentų rusiavimas į "gerus" ir "blogus" pagal galutini vidurki (≥ 5.0 - geras, kitaip - blogas):

	1000	10000	100000	1000000	10000000
1.	0.0004s	0.0015s	0.01s	0.11s	1.25s
2.	0.0002s	0.0013s	0.009s	0.11s	1.19s
3.	0.0003s	0.006s	0.01s	0.11s	1.28s
4.	0.0003s	0.0012s	0.01s	0.11s	1.29s
Vid.	0.0003s	0.0025s	0.00975s	0.11s	1.2525s

1.7.9.3 3. "Geru" ir "blogu" studentų atitinkamu failu generavimas:

	1000	10000	100000	1000000	10000000
1.	0.004s	0.03s	0.3s	4.03s	30.72s
2.	0.005s	0.03s	0.31s	3.18s	30.97s
3.	0.004s	0.03s	0.31s	3.14s	31.12s
4.	0.004s	0.03s	0.32s	3.1s	31.04s
Vid.	0.017s	0.03s	0.31s	3.3625s	30.9625s

1.7.9.4 4. Visos programos laikas:

	1000	10000	100000	1000000	10000000
1.	0.026s	0.09s	0.78s	8.83s	79.75s
2.	0.013s	0.09s	0.8s	8.12s	80.25s
3.	0.011s	0.099s	0.81s	8.04s	80.56s
4.	0.011s	0.088s	0.81s	7.95s	80.2s
Vid.	0.01525s	0.09175s	0.8s	8.235s	80.19s

1.7.9.5 Tyrimo 2 output ekrane:

1.7.10 ``

Kitas nuotraukas galima rasti repozitorijos assets aplanke (v0.4/Assets/...).

1.7.11 v1.0 (subrelease)

Tyrimai buvo atlikti su Visual Studio 2022 /O2 optimizacijos nustatymais.
 Tyrimo 1 failai buvo sukurti viena karta prieš bandymu pradžia.
 Testavimo metu testavimo sistemoje nebuvo įjungtos jokios kitos programos.

1.7.12 Tyrimas 1 - Aprasas

Atliktas tyrimas su trejais skirtingais STL konteineriais (Vektoriai, Sarasai (list) ir Deque).
 Atlikti 3 bandymai kiekvienam konteineriui ir apskaičiuotas vidurkis, kuris pateikiamas kuo tikslesnis (daugiausia 5 simboliai po kablelio).

Siame tyrime buvo ismatuota minetu konteineriu sparta, programai:

1. skaitant duomenis is failo,
2. rusiuojant duomenis didejimo tvarka pagal galutini vidurki (sort funkcija),
3. skirstant duomenis pagal galutini vidurki, sukuriant du naujus tokio pat tipo konteinerius ir naudojant move() funkcija, kad perkelti duomenis is originalaus konteinerio.

Rezultatai pateikiami sekundemis, suapvalinti (stengiamasi nevirsyti 3 skaitmenu po kablelio, taciau yra labai mazu duomenu).

1.7.13 Aktualus testavimo sistemos parametrai:**1.7.13.1 CPU**

Specification: 12th Gen Intel Core i7-12650H
Core count: 10
Thread count: 16
Hyperthreading: Not supported

1.7.13.2 RAM

Specification: DDR5-4800 (2400 MHz)
Memory slots used: 2 out of 2
Total memory: 16 GB

1.7.13.3 SSD

Specification: NVMe SAMSUNG MZVLQ1T0HBLB-00B00

Read/Write speed: 2300/1350 (MB/s)

Total capacity: 953 GB

1.7.14 Tyrimas 1 - Rezultatai

1.7.14.1 STD::VECTOR

Ivesciu sk.	Matavimo rodmuo	Test 1 (s)	Test 2 (s)	Test 3 (s)	Vidurkis (s)
1 000	Nuskaitymas	0.005	0.0048	0.0048	0.0049
	Rusiavimas (sort)	-	-	-	-
	Skirstymas	-	-	-	-
	Viskas	-	-	-	-
10 000	Nuskaitymas	0.045	0.047	0.047	0.046
	Rusiavimas (sort)	0.001	0.001	0.001	0.001
	Skirstymas	0.002	0.002	0.002	0.002
	Viskas	0.048	0.05	0.05	0.05
100 000	Nuskaitymas	0.47	0.48	0.46	0.47
	Rusiavimas (sort)	0.01	0.011	0.010	0.010
	Skirstymas	0.018	0.018	0.019	0.018
	Viskas	0.50	0.51	0.49	0.50
1 000 000	Nuskaitymas	4.57	4.60	4.63	4.60
	Rusiavimas (sort)	0.1180	0.117	0.12	0.12
	Skirstymas	0.19	0.19	0.19	0.19
	Viskas	4.88	4.90	4.94	4.91
10 000 000	Nuskaitymas	46.49	46.35	46.66	46.50
	Rusiavimas (sort)	1.11	1.12	1.19	1.14
	Skirstymas	2.28	2.45	2.38	2.37
	Viskas	49.89	49.92	50.26	50.01

1.7.14.2 STD::LIST

Ivesciu sk.	Matavimo rodmuo	Test 1 (s)	Test 2 (s)	Test 3 (s)	Vidurkis (s)
1 000	Nuskaitymas	0.005	0.006	0.005	0.005
	Rusiavimas (sort)	-	-	-	-
	Skirstymas	-	-	-	-
	Viskas	0.006	0.006	0.006	0.006
10 000	Nuskaitymas	0.047	0.046	0.044	0.046
	Rusiavimas (sort)	0.001	0.001	0.001	0.001
	Skirstymas	0.003	0.003	0.003	0.003
	Viskas	0.051	0.051	0.048	0.050
100 000	Nuskaitymas	0.450	0.459	0.462	0.457

Ivesciu sk.	Matavimo rodmuo	Test 1 (s)	Test 2 (s)	Test 3 (s)	Vidurkis (s)
	Rusiavimas (sort)	0.018	0.017	0.018	0.018
	Skirstymas	0.037	0.033	0.034	0.034
	Viskas	0.505	0.509	0.514	0.509
1 000 000	Nuskaitymas	4.501	4.538	4.791	4.610
	Rusiavimas (sort)	0.467	0.459	0.460	0.462
	Skirstymas	0.410	0.413	0.410	0.411
	Viskas	5.378	5.409	5.662	5.483
10 000 000	Nuskaitymas	47.784	45.986	46.689	46.820
	Rusiavimas (sort)	8.143	8.123	8.132	8.133
	Skirstymas	5.014	4.936	4.968	4.973
	Viskas	60.942	59.045	59.790	59.926

1.7.14.3 STD::DEQUE

Ivesciu sk.	Matavimo rodmuo	Test 1 (s)	Test 2 (s)	Test 3 (s)	Vidurkis (s)
1 000	Nuskaitymas	0.005	0.006	0.006	0.006
	Rusiavimas (sort)	-	-	-	-
	Skirstymas	0.001	-	-	-
	Viskas	0.006	0.006	0.007	0.006
10 000	Nuskaitymas	0.047	0.046	0.046	0.046
	Rusiavimas (sort)	0.001	0.001	0.002	0.001
	Skirstymas	0.004	0.003	0.003	0.003
	Viskas	0.052	0.050	0.051	0.051
100 000	Nuskaitymas	0.450	0.453	0.452	0.452
	Rusiavimas (sort)	0.014	0.014	0.014	0.014
	Skirstymas	0.024	0.024	0.024	0.024
	Viskas	0.488	0.490	0.490	0.489
1 000 000	Nuskaitymas	4.506	4.536	4.520	4.521
	Rusiavimas (sort)	0.174	0.172	0.178	0.175
	Skirstymas	0.235	0.240	0.245	0.240
	Viskas	4.915	4.948	4.944	4.936
10 000 000	Nuskaitymas	45.918	45.904	48.004	46.609
	Rusiavimas (sort)	1.755	1.818	1.813	1.795
	Skirstymas	3.309	3.179	3.211	3.233
	Viskas	50.983	50.902	53.027	51.637

1.7.15 v1.0 (optimizuota)

Tyrimai buvo atlikti su Visual Studio 2022 /O2 optimizacijos nustatymais.
 Tyrimo 2 failai buvo sukurti viena karta prieš bandymu pradzia.
 Testavimo metu testavimo sistemoje nebuvo ijungtos jokios kitos programos.

1.7.16 Tyrimas 2 - Aprasas

Atliktas tyrimas su trejais skirtingais STL konteineriais (Vektoriai, Sarasai (list) ir Deque).

Atlikti 3 bandymai kiekvienam konteineriui ir apskaiciuotas spartos vidurkis.

Kadangi spartos duomenys (skaiciai) buvo gan mazi, nebuvo apvalinta.

Siame tyrime buvo ismatuota minetu konteineriu sparta, programai skirstant duomenis pagal 4 strategijas:

1. Pirmajame tyrime naudota strategija - studentu konteinerio duomenu isvedimas i du naujus konteinerius naudojant `std::move()`, tuomet istrinimas tusciu duomenu is originalaus konteinerio;
2. Studentu konteinerio duomenu isvedimas i du naujus konteinerius naudojant `push_back()`;
3. Studentu konteinerio duomenu isvedimas i viena nauja ('blogu' studentu) konteineri naudojant `std::move()` ir tuo pat metu istrinimas is originalaus konteinerio, naudojant iteratorius;
4. 3 strategija, optimizuota naudojant `std::stable_partition()` ir `std::move()`.

2 Hierarchijos Indeksas

2.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

Timer	14
Vector< T >	14
Zmogus	15
Studentas	13

3 Klasės Indeksas

3.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

Studentas	13
Timer	14
Vector< T >	14
Zmogus	15

4 Failo Indeksas

4.1 Failai

Visų dokumentuotų failų sąrašas su trumpais aprašymais:

functions.h	16
mylib.h	16
Studentas.h	16
timer.h	20

5 Klasės Dokumentacija

5.1 Studentas Klasė

Paveldimumo diagrama Studentas:

Bendradarbiavimo diagrama Studentas:

Vieši Metodai

- **Studentas** (std::string v, std::string p)
- **Studentas** (std::istream &in)
- **Studentas** (const [Studentas](#) &other)
- **Studentas** ([Studentas](#) &&other) noexcept
- [Studentas](#) & **operator=** (const [Studentas](#) &other)
- [Studentas](#) & **operator=** ([Studentas](#) &&other) noexcept
- const [Vector](#)< double > & **getPazymiai** () const
- int **getPazymiaiSize** () const
- double **getEgzaminas** () const
- double **getGalutinis** (double*)(const [Vector](#)< double > &)=calc_mediana) const
- std::istream & **read** (std::istream &input)
- void **addPazymys** (const double &pazymys)
- void **setGalutinis** (const double &galutinis)
- void **setEgzaminas** (const double &egzaminas)
- void **setRandVardas** ()
- void **setRandPavarde** (std::string &vardas)
- void **setRandEgzaminas** ()
- void **setRandPazymiai** ()
- void **paz_input** ()
- void **egz_input** ()

Vieši Metodai inherited from [Zmogus](#)

- **Zmogus** (std::string v, std::string p)
- **Zmogus** (std::istream &in)
- std::string **getVardas** () const
- std::string **getPavarde** () const
- void **setVardas** (std::string v)
- void **setPavarde** (std::string p)
- void **vardas_input** ()
- void **pavarde_input** ()

Draugai

- std::istream & **operator>>** (std::istream &in, [Studentas](#) &s)
- std::ostream & **operator<<** (std::ostream &out, const [Studentas](#) &s)

Additional Inherited Members

Apsaugoti Atributai inherited from [Zmogus](#)

- std::string **vardas_**
- std::string **pavarde_**

5.1.1 Metodų Dokumentacija

read()

```
std::istream & Studentas::read (
    std::istream & input) [virtual]
```

Realizuoja [Zmogus](#).

Dokumentacija šiai klasei sugeneruota iš šių failų:

- Studentas.h
- Studentas.cpp

5.2 Timer Klasė

Vieši Metodai

- void **reset** ()
- double **elapsed** () const

Dokumentacija šiai klasei sugeneruota iš šio failo:

- timer.h

5.3 Vector< T > Klasė Šablonas

Vieši Tipai

- using **size_type** = std::size_t
- using **ref** = T&
- using **const_ref** = const T&
- using **it** = T*
- using **const_it** = const T*
- using **reverse_it** = std::reverse_iterator<it>
- using **const_reverse_it** = std::reverse_iterator<const_it>

Vieši Metodai

- **Vector** (size_type count)
- **Vector** (size_type count, const T &value)
- **Vector** (std::initializer_list< T > init)
- **Vector** (const [Vector](#) &other)
- **Vector** ([Vector](#) &&other) noexcept
- void **assign** (size_type count, const T &value)
- template<std::ranges::input_range R>
void **assign_range** (R &&rg)
- ref **at** (size_type pos)
- const_ref **at** (size_type pos) const
- ref **front** ()
- const_ref **front** () const
- ref **back** ()
- const_ref **back** () const
- T * **data** ()
- const T * **data** () const
- it **begin** () noexcept
- it **end** () noexcept
- const_it **begin** () const noexcept
- const_it **end** () const noexcept
- const_it **cbegin** () const noexcept
- const_it **cend** () const noexcept

- `reverse_it` **rbegin** () noexcept
- `reverse_it` **rend** () noexcept
- `const_reverse_it` **rbegin** () const noexcept
- `const_reverse_it` **rend** () const noexcept
- `const_reverse_it` **crbegin** () const noexcept
- `const_reverse_it` **crend** () const noexcept
- `bool` **empty** () const
- `size_type` **size** () const
- `size_type` **max_size** () const
- `size_type` **capacity** () const
- `void` **reserve** (size_type new_capacity)
- `void` **shrink_to_fit** ()
- `void` **clear** ()
- `it` **insert** (const_it pos, const T &value)
- `it` **insert** (const_it pos, T &&value)
- `it` **insert** (const_it pos, std::initializer_list< T > ilist)
- `template<class InputIt>`
 `it` **insert** (const_it pos, InputIt first, InputIt last)
- `template<std::ranges::input_range R>`
 `constexpr it` **insert_range** (const_it pos, R &&rg)
- `void` **push_back** (const T &value)
- `void` **push_back** (T &&value)
- `void` **pop_back** ()
- `void` **resize** (size_type count)
- `void` **resize** (size_type count, const T &value)
- `template<class... Args>`
 `it` **emplace** (const_it pos, Args &&... args)
- `template<class... Args>`
 `ref` **emplace_back** (Args &&... args)
- `it` **erase** (const_it pos)
- `it` **erase** (const_it first, const_it last)
- `constexpr void` **swap** (`Vector` &other) noexcept
- `Vector` & **operator=** (const `Vector` &other)
- `Vector` & **operator=** (`Vector` &&other) noexcept
- `ref` **operator[]** (size_type pos)
- `const_ref` **operator[]** (size_type pos) const

Dokumentacija šiai klasei sugeneruota iš šio failo:

- Vector.h

5.4 Zmogus Klasė

Paveldimumo diagrama Zmogus:

Vieši Metodai

- **Zmogus** (std::string v, std::string p)
- **Zmogus** (std::istream &in)
- std::string **getVardas** () const
- std::string **getPavarde** () const
- void **setVardas** (std::string v)
- void **setPavarde** (std::string p)
- virtual std::istream & **read** (std::istream &in)=0
- void **vardas_input** ()
- void **pavarde_input** ()

Apsaugoti Atributai

- std::string vardas_
- std::string pavarde_

Dokumentacija šiai klasei sugeneruota iš šių failų:

- Studentas.h
- Studentas.cpp

6 Failo Dokumentacija

6.1 functions.h

```
00001 #pragma once
00002 #include "mylib.h"
00003
00004 void menu(int& choiceMenu);
00005
00006 int integer_input_validation(int lowEnd, int highEnd, std::string optionalPrompt = ""); //if highEnd =
-1, no highEnd used
00007
00008 std::string string_input_validation(std::string optionalPrompt = "");
00009
00010 void student_file_generator(int nStud, int nPaz);
```

6.2 mylib.h

```
00001 #include <iostream>
00002 #include <iomanip>
00003 #include <string>
00004 #include <vector>
00005 #include <algorithm>
00006 #include <random>
00007 #include <stdlib.h>
00008 #include <fstream>
00009 #include <list>
00010 #include <deque>
00011 #include <sstream>
00012 #include <filesystem>
00013 #include <chrono>
00014 #include "timer.h"
00015
00016 using std::cout;
00017 using std::cin;
00018
00019 namespace fs = std::filesystem;
```

6.3 Studentas.h

```
00001 #pragma once
00002 #include "mylib.h"
00003 #include "Vector.h"
00004
00005 double calc_vidurkis(const Vector<double>&); //Calculate vidurkis
00006 double calc_mediana(const Vector<double>&); //Calculate mediana
00007
00008 class Zmogus {
00009 protected:
00010     std::string vardas_;
00011     std::string pavarde_;
00012 public:
00013     Zmogus() = default;
00014     Zmogus(std::string v, std::string p) : vardas_(v), pavarde_(p) {}
00015     Zmogus(std::istream& in) { in » vardas_ » pavarde_; }
00016     virtual ~Zmogus() = default;
00017
00018     inline std::string getVardas() const { return vardas_; }
00019     inline std::string getPavarde() const { return pavarde_; }
00020
00021     void setVardas(std::string v) { vardas_ = v; }
00022     void setPavarde(std::string p) { pavarde_ = p; }
00023
00024     virtual std::istream& read(std::istream& in) = 0;
00025
00026     void vardas_input();
00027     void pavarde_input();
```

```

00028 };
00029
00030 class Studentas : public Zmogus {
00031 private:
00032     Vector<double> paz_; // nd pazymiai
00033     double egzaminas_ = 0;
00034     double galutinis_ = 0;
00035
00036 public:
00037     Studentas() = default;
00038     Studentas(std::string v, std::string p) : Zmogus(v, p) {}
00039     Studentas(std::istream& in);
00040     ~Studentas() override = default;
00041
00042     Studentas(const Studentas& other); //copy
00043     Studentas(Studentas&& other) noexcept; //move
00044
00045     Studentas& operator=(const Studentas& other); //copy assign
00046     Studentas& operator=(Studentas&& other) noexcept; //move assign
00047
00048     friend std::istream& operator>(std::istream& in, Studentas& s); //input
00049     friend std::ostream& operator<(std::ostream& out, const Studentas& s); //output
00050
00051     const Vector<double>& getPazymiai() const { return paz_; };
00052     int getPazymiaiSize() const { return paz_.size(); };
00053     double getEgzaminas() const { return egzaminas_; };
00054     double getGalutinis(double (*)(const Vector<double>&) = calc_mediana) const; //returns apdorotas
    galutinis
00055     std::istream& read(std::istream& input);
00056
00057     void addPazymys(const double& pazymys) { paz_.push_back(pazymys); };
00058     void setGalutinis(const double& galutinis) { galutinis_ = galutinis; };
00059     void setEgzaminas(const double& egzaminas) { egzaminas_ = egzaminas; };
00060
00061     void setRandVardas();
00062     void setRandPavarde(std::string& vardas);
00063     void setRandEgzaminas();
00064     void setRandPazymiai();
00065
00066     //Input funkcijos
00067     void paz_input();
00068     void egz_input();
00069
00070 };
00071
00072 //Perskaityti egzistuojanti studentu duomenu faila ir irasyti i stl konteineri
00073 template<typename StudentaiContainer>
00074 void read_file(const std::string& filename, StudentaiContainer& studentai) {
00075     fs::path filePath = filename;
00076
00077     try {
00078         //CHECK IMPORTANT EXCEPTIONS
00079
00080         if (filePath.extension() != ".txt") {
00081             throw std::runtime_error("\n---KLaida: Failas " + filename + " turi baigtis '.txt'---\n");
00082         }
00083
00084         if (!fs::exists(filename)) {
00085             throw std::runtime_error("\n---KLaida: Failas " + filename + " neegzistuoja---\n");
00086         }
00087
00088         //Open file
00089         std::fstream fin(filename, std::ios::in);
00090
00091         if (!fin.is_open()) {
00092             throw std::runtime_error("\n---KLaida: Failo " + filename + " nepavyko atidaryti---\n");
00093         }
00094
00095         //Read file
00096         std::string curr_eil;
00097
00098         fin.ignore(INT32_MAX, '\n');
00099
00100         while (std::getline(fin, curr_eil)) {
00101             std::istringstream iss(curr_eil);
00102
00103             //read student
00104             Studentas temp_studentas(iss);
00105
00106             studentai.push_back(temp_studentas);
00107         }
00108
00109         fin.close();
00110     }
00111     catch (const std::exception& e) {
00112         std::cerr << e.what() << '\n';
00113     }

```

```

00114 }
00115
00116 //Irasyti studentu duomenis i faila / sukurti nauja faila su duomenimis
00117 template<typename StudentaiContainer>
00118 void write_studentai(const std::string& filename, StudentaiContainer& studentai) {
00119     //write to file
00120     std::ofstream fout(filename);
00121
00122     fout << '\n' << std::setw(15) << std::left << "Vardas" << std::setw(15) << std::left << "Pavarde"
00123     << std::setw(15) << std::left << "Galutinis (Vid.)" << std::setw(15) << std::left <<
00124     "Galutinis (Med.)"
00125     << std::setw(15) << std::left << "Egz." << '\n';
00126     fout << "-----\n";
00127     for (const auto& s : studentai) {
00128         fout << s << '\n';
00129     }
00130     fout.close();
00131 }
00132
00133 template<typename StudentaiContainer>
00134 void choice_sort(StudentaiContainer& studentai, int choice)
00135 {
00136     std::sort(studentai.begin(), studentai.end(),
00137         [choice](const Studentas& a, const Studentas& b)
00138         {
00139             if (choice == 1)
00140             {
00141                 return a.getVardas() < b.getVardas();
00142             }
00143             else if (choice == 2)
00144             {
00145                 return a.getPavarde() < b.getPavarde();
00146             }
00147             else if (choice == 3)
00148             {
00149                 return a.getGalutinis(calc_vidurkis) >
00150                     b.getGalutinis(calc_vidurkis);
00151             }
00152             else
00153             {
00154                 return a.getGalutinis() > b.getGalutinis();
00155             }
00156         });
00157 }
00158
00159 //Sort vector
00160 void vidurkis_sort(Vector<Studentas>& studentai);
00161
00162 //Sort deque
00163 void vidurkis_sort(std::deque<Studentas>& studentai);
00164
00165 //Sort list
00166 void vidurkis_sort(std::list<Studentas>& studentai);
00167
00168 void split_file_generator(const std::string& filename, Vector<Studentas>& studentai);
00169
00170 void student_split(std::string filename, Vector<Studentas>& studentai);
00171
00172
00173 void testing_v04_1(int nStud);
00174
00175 void testing_v04_2(std::string filename);
00176
00177
00178
00179
00180
00181 template <typename StudentaiContainer>
00182 void student_split_strategy0(StudentaiContainer& studentai) {
00183     Vector<Studentas> studGeri, studBlogi;
00184     for (auto& s : studentai) {
00185         if (s.getGalutinis(calc_vidurkis) < 5.0)
00186             studBlogi.push_back(std::move(s));
00187         else
00188             studGeri.push_back(std::move(s));
00189     }
00190     studentai.clear();
00191 }
00192
00193 template <typename StudentaiContainer>
00194 void student_split_strategy1(StudentaiContainer& studentai) {
00195     StudentaiContainer studBlogi, studGeri;
00196     for (const auto& s : studentai) {
00197         if (s.getGalutinis(calc_vidurkis) < 5.0)
00198             studBlogi.push_back(s);
00199         else
00200             studGeri.push_back(s);
00201     }

```

```

00202 }
00203
00204 template <typename StudentaiContainer>
00205 void student_split_strategy2(StudentaiContainer& studentai) {
00206     StudentaiContainer studBlogi;
00207
00208     auto it = studentai.begin();
00209     while (it != studentai.end()) {
00210         if (it->getGalutinis(calc_vidurkis) < 5.0) {
00211             studBlogi.push_back(std::move(*it));
00212             //push iterator and remove studBlogi from studentai
00213             it = studentai.erase(it);
00214         }
00215         else {
00216             ++it;
00217         }
00218     }
00219 }
00220
00221 template <typename StudentaiContainer>
00222 void student_split_strategy3(StudentaiContainer& studentai) {
00223     //reorder - good first, bad after
00224     auto partition_point = std::stable_partition(
00225         studentai.begin(), studentai.end(),
00226         [](const Studentas& s) { return s.getGalutinis(calc_vidurkis) >= 5.0; }
00227     );
00228     //move
00229     StudentaiContainer studBlogi;
00230     for (auto it = partition_point; it != studentai.end(); ++it) {
00231         studBlogi.push_back(std::move(*it));
00232     }
00233 }
00234
00235
00236 //Initial test of containers: reading, sorting + splitting students, writing
00237 template<typename StudentaiContainer>
00238 void test1_containers(int nStud) {
00239     std::string filename = "studentai" + std::to_string(nStud) + ".txt";
00240
00241     cout << "\n---TESTAVIMAS SU " << nStud << " STUDENTU IVESCIU---\n\n";
00242
00243     //reading
00244     Timer t;
00245     StudentaiContainer studentai;
00246
00247     read_file<StudentaiContainer>(filename, studentai);
00248
00249     double read_t = t.elapsed();
00250     cout << "Duomenų nuskaitymas iš failo " << filename << " užtruko: " << read_t << " s\n\n";
00251     system("pause");
00252
00253     //sort by galutinisVid
00254     Timer t1;
00255     vidurkis_sort(studentai);
00256     double sort_t = t1.elapsed();
00257     cout << "\nStudentu sort() " << filename << " užtruko: " << sort_t << " s\n\n";
00258     system("pause");
00259
00260     //split students
00261     Timer t2;
00262     student_split_strategyold(studentai);
00263     double split_t = t2.elapsed();
00264     cout << "\nStudentu paskirstymas į 'gerus' ir 'blogus' " << filename << " užtruko: " << split_t << " s\n\n";
00265     system("pause");
00266
00267     //all results for nStud
00268
00269     cout << "\n\n---REZULTATAI SU " << nStud << " STUDENTU IVESCIU---\n\n";
00270     cout << "Nuskaitymas: " << read_t << " s\n";
00271     cout << "Rusaviimas (sort): " << sort_t << " s\n";
00272     cout << "Skirstymas: " << split_t << " s\n";
00273     cout << "Viskas: " << (read_t + sort_t + split_t) << " s\n";
00274
00275 }
00276
00277 //Test every num of students with one type of container
00278 template <typename StudentaiContainer>
00279 void run_test_containers() {
00280     test1_containers<StudentaiContainer>(1000);
00281     test1_containers<StudentaiContainer>(10000);
00282     test1_containers<StudentaiContainer>(100000);
00283     test1_containers<StudentaiContainer>(1000000);
00284     test1_containers<StudentaiContainer>(10000000);
00285 }
00286
00287 //Test every num of students with one type of container

```

```

00288 template <typename StudentaiContainer>
00289 void test1_splitting(int nStud) {
00290     std::string filename = "studentai" + std::to_string(nStud) + ".txt";
00291
00292     //read
00293     StudentaiContainer studentai_original;
00294     read_file<StudentaiContainer>(filename, studentai_original);
00295
00296     cout << "\n---STRATEGIJA 0 (PIRMINE)---\n\n";
00297
00298     //copy and sort
00299     StudentaiContainer studentai0 = studentai_original;
00300     vidurkis_sort(studentai0);
00301
00302     //split students and time
00303     Timer t;
00304     student_split_strategy0(studentai0);
00305     double split_t = t.elapsed();
00306     cout << "\nStudentu paskirstymas i 'gerus' ir 'blogus' " << filename << " uztruko: " << split_t << "
s\n\n";
00307     system("pause");
00308
00309     cout << "\n---STRATEGIJA 1---\n\n";
00310
00311     //copy and sort
00312     StudentaiContainer studentai1 = studentai_original;
00313     vidurkis_sort(studentai1);
00314
00315     //split students and time
00316     Timer t1;
00317     student_split_strategy1(studentai1);
00318     double split_t1 = t1.elapsed();
00319     cout << "\nStudentu paskirstymas i 'gerus' ir 'blogus' " << filename << " uztruko: " << split_t1 << "
s\n\n";
00320     system("pause");
00321
00322     cout << "\n---STRATEGIJA 2---\n\n";
00323
00324     //copy and sort
00325     StudentaiContainer studentai2 = studentai_original;
00326     vidurkis_sort(studentai2);
00327
00328     //split students and time
00329     Timer t2;
00330     student_split_strategy2(studentai2);
00331     double split_t2 = t2.elapsed();
00332     cout << "\nStudentu paskirstymas i 'gerus' ir 'blogus' " << filename << " uztruko: " << split_t2 << "
s\n\n";
00333     system("pause");
00334
00335     cout << "\n---STRATEGIJA 3---\n\n";
00336
00337     //copy and sort
00338     StudentaiContainer studentai3 = studentai_original;
00339     vidurkis_sort(studentai3);
00340
00341     //split students and time
00342     Timer t3;
00343     student_split_strategy3(studentai3);
00344     double split_t3 = t3.elapsed();
00345     cout << "\nStudentu paskirstymas i 'gerus' ir 'blogus' " << filename << " uztruko: " << split_t3 << "
s\n\n";
00346     system("pause");
00347     system("cls");
00348 }
00349
00350 template <typename StudentaiContainer>
00351 void run_test_splitting() {
00352     test1_splitting<StudentaiContainer>(1000);
00353     test1_splitting<StudentaiContainer>(10000);
00354     test1_splitting<StudentaiContainer>(100000);
00355     test1_splitting<StudentaiContainer>(1000000);
00356     test1_splitting<StudentaiContainer>(10000000);
00357 }
00358
00359 void testROF();

```

6.4 timer.h

```

00001 #pragma once
00002 #include <chrono>
00003 class Timer {
00004 private:
00005     std::chrono::time_point<std::chrono::high_resolution_clock> start;
00006 public:

```

```

00007     Timer() : start{ std::chrono::high_resolution_clock::now() } {}
00008     void reset() {
00009         start = std::chrono::high_resolution_clock::now();
00010     }
00011     double elapsed() const {
00012         return std::chrono::duration<double>(std::chrono::high_resolution_clock::now() -
start).count();
00013     }
00014 };

```

6.5 Vector.h

```

00001 #pragma once
00002 #include <stdexcept>
00003 #include <ranges>
00004 #include <limits>
00005 #include <algorithm>
00006 #include <compare>
00007 #include <cassert>
00008 #include "timer.h"
00009
00010 template<typename T>
00011 class Vector {
00012 private:
00013     T* data_;
00014     size_t size_;
00015     size_t capacity_;
00016
00017 public:
00018     using size_type = std::size_t;
00019     using ref = T&; //reference
00020     using const_ref = const T&;
00021     using it = T*; //iterator
00022     using const_it = const T*;
00023     using reverse_it = std::reverse_iterator<it>;
00024     using const_reverse_it = std::reverse_iterator<const_it>;
00025
00026     //member functions
00027     Vector();
00028     Vector(size_type count);
00029     Vector(size_type count, const T& value);
00030     Vector(std::initializer_list<T> init);
00031     Vector(const Vector& other);
00032     Vector(Vector&& other) noexcept;
00033
00034     ~Vector();
00035
00036     void assign(size_type count, const T& value);
00037     template<std::ranges::input_range R>
00038     void assign_range(R&& rg);
00039
00040     //element access
00041
00042     ref at(size_type pos);
00043     const_ref at(size_type pos) const;
00044
00045     ref front();
00046     const_ref front() const;
00047
00048     ref back();
00049     const_ref back() const;
00050
00051     T* data();
00052     const T* data() const;
00053
00054
00055
00056     // iterators
00057     it begin() noexcept { return data_; }
00058     it end() noexcept { return data_ + size_; }
00059
00060     const_it begin() const noexcept { return data_; }
00061     const_it end() const noexcept { return data_ + size_; }
00062
00063     const_it cbegin() const noexcept { return begin(); }
00064     const_it cend() const noexcept { return end(); }
00065
00066     // reverse iterators
00067     reverse_it rbegin() noexcept { return reverse_it(end()); }
00068     reverse_it rend() noexcept { return reverse_it(begin()); }
00069
00070     const_reverse_it rbegin() const noexcept { return const_reverse_it(end()); }
00071     const_reverse_it rend() const noexcept { return const_reverse_it(begin()); }
00072
00073     const_reverse_it crbegin() const noexcept { return rbegin(); }

```

```

00074     const_reverse_it crend() const noexcept { return rend(); }
00075
00076     //capacity
00077
00078     bool empty() const { return size_ == 0; }; //getter
00079     size_type size() const { return size_; } //getter
00080     size_type max_size() const { return std::numeric_limits<size_type>::max(); } //getter
00081     size_type capacity() const { return capacity_; } //getter
00082
00083     void reserve(size_type new_capacity);
00084     void shrink_to_fit();
00085
00086     //modifiers
00087
00088     void clear();
00089     it insert(const_it pos, const T& value);
00090     it insert(const_it pos, T&& value);
00091     it insert(const_it pos, std::initializer_list<T> ilist);
00092     template<class InputIt>
00093     it insert(const_it pos, InputIt first, InputIt last);
00094     template<std::ranges::input_range R>
00095     constexpr it insert_range(const_it pos, R&& rg);
00096     void push_back(const T& value);
00097     void push_back(T&& value);
00098     void pop_back();
00099     void resize(size_type count);
00100     void resize(size_type count, const T& value);
00101     template<class... Args>
00102     it emplace(const_it pos, Args&&... args);
00103     template<class... Args>
00104     ref emplace_back(Args&&... args);
00105     it erase(const_it pos);
00106     it erase(const_it first, const_it last);
00107     constexpr void swap(Vector& other) noexcept;
00108
00109     // operators
00110
00111     Vector& operator=(const Vector& other);
00112
00113     Vector& operator=(Vector&& other) noexcept;
00114
00115     ref operator[](size_type pos);
00116     const_ref operator[](size_type pos) const;
00117 };
00118
00119 // constructors
00120
00121 template<typename T>
00122 Vector<T>::Vector()
00123 :
00124     data_(nullptr),
00125     size_(0),
00126     capacity_(0)
00127 {
00128 }
00129
00130 template<typename T>
00131 Vector<T>::Vector(size_type count)
00132 :
00133     data_(new T[count]),
00134     size_(count),
00135     capacity_(count)
00136 {
00137 }
00138
00139 template<typename T>
00140 Vector<T>::Vector(size_type count, const T& value)
00141 :
00142     data_(new T[count]),
00143     size_(count),
00144     capacity_(count)
00145 {
00146     assign(count, value);
00147 }
00148
00149 template<typename T>
00150 Vector<T>::Vector(std::initializer_list<T> init) {
00151     size_ = init.size();
00152     capacity_ = init.size();
00153     data_ = new T[capacity_];
00154
00155     size_type i = 0;
00156     for (const auto& v : init)
00157         data_[i++] = v;
00158 }
00159
00160 template<typename T>

```

```

00161 Vector<T>::Vector(const Vector& other)
00162 :
00163     data_(nullptr),
00164     size_(other.size_),
00165     capacity_(other.capacity_)
00166 {
00167     if (capacity_ > 0)
00168     {
00169         data_ = new T[capacity_];
00170
00171         for (size_type i = 0; i < size_; ++i)
00172             data_[i] = other.data_[i];
00173     }
00174 }
00175
00176 template<typename T>
00177 Vector<T>::Vector(Vector&& other) noexcept
00178 :
00179     data_(other.data_),
00180     size_(other.size_),
00181     capacity_(other.capacity_)
00182 {
00183     other.data_ = nullptr;
00184     other.size_ = 0;
00185     other.capacity_ = 0;
00186 }
00187
00188 // destructor
00189
00190 template<typename T>
00191 Vector<T>::~Vector() {
00192     delete[] data_;
00193 }
00194
00195 //operators
00196
00197 template<typename T>
00198 Vector<T>& Vector<T>::operator=(const Vector& other)
00199 {
00200     if (this == &other)
00201         return *this;
00202
00203     T* new_data = nullptr;
00204
00205     if (other.capacity_ > 0)
00206     {
00207         new_data = new T[other.capacity_];
00208
00209         for (size_type i = 0; i < other.size_; ++i)
00210             new_data[i] = other.data_[i];
00211     }
00212
00213     delete[] data_;
00214
00215     data_ = new_data;
00216     size_ = other.size_;
00217     capacity_ = other.capacity_;
00218
00219     return *this;
00220 }
00221
00222 template<typename T>
00223 Vector<T>& Vector<T>::operator=(Vector&& other) noexcept
00224 {
00225     if (this == &other)
00226         return *this;
00227
00228     delete[] data_;
00229
00230     data_ = other.data_;
00231     size_ = other.size_;
00232     capacity_ = other.capacity_;
00233
00234     other.data_ = nullptr;
00235     other.size_ = 0;
00236     other.capacity_ = 0;
00237
00238     return *this;
00239 }
00240
00241 template<typename T>
00242 Vector<T>::ref Vector<T>::operator[](size_type pos) {
00243     return data_[pos];
00244 }
00245
00246 template<typename T>
00247 Vector<T>::const_ref Vector<T>::operator[](size_type pos) const {

```

```

00248     return data_[pos];
00249 }
00250
00251 //non-member operators
00252
00253 template<typename T>
00254 bool operator==(const Vector<T>& a, const Vector<T>& b)
00255 {
00256     if (a.size() != b.size())
00257         return false;
00258
00259     for (size_t i = 0; i < a.size(); ++i) {
00260         if (!a[i] == b[i]) return false;
00261     }
00262
00263     return true;
00264 }
00265
00266 template<typename T>
00267 bool operator!=(const Vector<T>& a, const Vector<T>& b)
00268 {
00269     return !(a == b);
00270 }
00271
00272 template<typename T>
00273 bool operator<(const Vector<T>& a, const Vector<T>& b)
00274 {
00275     return std::lexicographical_compare(
00276         a.begin(), a.end(),
00277         b.begin(), b.end()
00278     );
00279 }
00280
00281 template<typename T>
00282 bool operator>(const Vector<T>& a, const Vector<T>& b)
00283 {
00284     return b < a;
00285 }
00286
00287 template<typename T>
00288 bool operator<=(const Vector<T>& a, const Vector<T>& b)
00289 {
00290     return !(b < a);
00291 }
00292
00293 template<typename T>
00294 bool operator>=(const Vector<T>& a, const Vector<T>& b)
00295 {
00296     return !(a < b);
00297 }
00298
00299 template<typename T>
00300 auto operator<=>(const Vector<T>& a, const Vector<T>& b)
00301 {
00302     return std::lexicographical_compare_three_way(
00303         a.begin(), a.end(),
00304         b.begin(), b.end()
00305     );
00306 }
00307
00308 //non-member functions
00309
00310 template<typename T>
00311 typename Vector<T>::size_type erase(Vector<T>& v, const T& value)
00312 {
00313     auto new_end = std::remove(v.begin(), v.end(), value);
00314     auto removed = v.end() - new_end;
00315
00316     v.erase(new_end, v.end());
00317
00318     return removed;
00319 }
00320
00321 template<typename T, class Pred>
00322 typename Vector<T>::size_type erase_if(Vector<T>& v, Pred pred)
00323 {
00324     auto new_end = std::remove_if(v.begin(), v.end(), pred);
00325
00326     size_t removed = v.end() - new_end;
00327
00328     v.erase(new_end, v.end());
00329
00330     return removed;
00331 }
00332
00333 // member functions
00334

```

```

00335 template<typename T>
00336 void Vector<T>::assign(size_type count, const T& value) {
00337     if (count > capacity_) {
00338         delete[] data_;
00339
00340         data_ = new T[count];
00341         capacity_ = count;
00342     }
00343
00344     for (size_type i = 0; i < count; i++) {
00345         data_[i] = value;
00346     }
00347     size_ = count;
00348 }
00349
00350 template<typename T>
00351 template<std::ranges::input_range R>
00352 void Vector<T>::assign_range(R&& rg) {
00353     clear();
00354
00355     for (auto&& value : rg) {
00356         push_back(value);
00357     }
00358 }
00359
00360 // element access
00361
00362 template<typename T>
00363 typename Vector<T>::ref Vector<T>::at(size_type pos) {
00364     if (pos >= size_ || empty()) throw std::out_of_range("Vector::at");
00365     return data_[pos];
00366 }
00367
00368 template<typename T>
00369 typename Vector<T>::const_ref Vector<T>::at(size_type pos) const {
00370     if (pos >= size_ || empty()) throw std::out_of_range("Vector::at");
00371     return data_[pos];
00372 }
00373
00374 template<typename T>
00375 typename Vector<T>::ref Vector<T>::front() {
00376     return *begin();
00377 }
00378
00379 template<typename T>
00380 typename Vector<T>::const_ref Vector<T>::front() const {
00381     return *begin();
00382 }
00383
00384 template<typename T>
00385 typename Vector<T>::ref Vector<T>::back() {
00386     return *(end() - 1);
00387 }
00388
00389 template<typename T>
00390 typename Vector<T>::const_ref Vector<T>::back() const {
00391     return *(end() - 1);
00392 }
00393
00394 T* Vector<T>::data() {
00395     return data_;
00396 }
00397
00398 const T* Vector<T>::data() const {
00399     return data_;
00400 }
00401
00402 // capacity
00403
00404 template<typename T>
00405 void Vector<T>::reserve(size_type new_capacity) {
00406     if (new_capacity <= capacity_)
00407         return;
00408
00409     T* new_data = new T[new_capacity];
00410     for (size_type i = 0; i < size_; i++) {
00411         new_data[i] = data_[i];
00412     }
00413     delete[] data_;
00414     data_ = new_data;
00415     capacity_ = new_capacity;
00416 }

```

```

00422
00423 template<typename T>
00424 void Vector<T>::shrink_to_fit()
00425 {
00426     if (size_ == capacity_) return;
00427
00428     T* new_data = nullptr;
00429
00430     if (size_ > 0)
00431     {
00432         new_data = new T[size_];
00433
00434         for (size_type i = 0; i < size_; ++i)
00435             new_data[i] = std::move(data_[i]);
00436     }
00437
00438     delete[] data_;
00439     data_ = new_data;
00440     capacity_ = size_;
00441 }
00442
00443 // modifiers
00444
00445 template<typename T>
00446 template<class... Args>
00447 typename Vector<T>::it Vector<T>::emplace(const_it pos, Args&&... args)
00448 {
00449     size_type index = pos - begin();
00450
00451     if (size_ == capacity_)
00452         reserve(capacity_ ? capacity_ * 2 : 1);
00453
00454     std::move_backward(begin() + index, end(), end() + 1);
00455
00456     data_[index] = T(std::forward<Args>(args)...);
00457
00458     ++size_;
00459     return begin() + index;
00460 }
00461
00462 template<typename T>
00463 template<class... Args>
00464 typename Vector<T>::ref Vector<T>::emplace_back(Args&&... args) {
00465     emplace(end(), std::forward<Args>(args)...);
00466
00467     return back();
00468 }
00469
00470 template<typename T>
00471 void Vector<T>::clear()
00472 {
00473     for (size_type i = 0; i < size_; ++i)
00474         data_[i].~T();
00475
00476     size_ = 0;
00477 }
00478
00479 template<typename T>
00480 typename Vector<T>::it Vector<T>::insert(const_it pos, const T& value) {
00481     if (pos < begin() || pos > end()) throw std::out_of_range("Vector::insert");
00482
00483     size_type index = pos - begin(); //del reserve()
00484
00485     if (size_ == capacity_) reserve(capacity_ == 0 ? 1 : capacity_ * 2);
00486
00487     std::move_backward(begin() + index, end(), end() + 1);
00488
00489     data_[index] = value;
00490     ++size_;
00491
00492     return begin() + index;
00493 }
00494
00495 template<typename T>
00496 template<class InputIt>
00497 typename Vector<T>::it
00498 Vector<T>::insert(const_it pos, InputIt first, InputIt last)
00499 {
00500     size_type index = pos - begin();
00501
00502     size_type count = std::distance(first, last);
00503
00504     if (size_ + count > capacity_)
00505         reserve(std::max(capacity_ * 2, size_ + count));
00506
00507     std::move_backward(begin() + index, end(), end() + count);
00508

```

```

00509     size_type i = index;
00510     for (auto it = first; it != last; ++it)
00511         data_[i++] = *it;
00512
00513     size_ += count;
00514
00515     return begin() + index;
00516 }
00517
00518 template<typename T>
00519 typename Vector<T>::it Vector<T>::insert(const_it pos, T&& value) {
00520     if (pos < begin() || pos > end()) throw std::out_of_range("Vector::insert");
00521
00522     size_type index = pos - begin();
00523
00524     if (size_ == capacity_) reserve(capacity_ == 0 ? 1 : capacity_ * 2);
00525
00526     std::move_backward(begin() + index, end(), end() + 1);
00527
00528     data_[index] = std::move(value);
00529     size_++;
00530
00531     return begin() + index;
00532 }
00533
00534 template<typename T>
00535 typename Vector<T>::it Vector<T>::insert(const_it pos, std::initializer_list<T> ilist) {
00536     size_type index = pos - begin();
00537     size_type count = ilist.size();
00538
00539     if (size_ + count > capacity_)
00540         reserve(std::max(capacity_ * 2, size_ + count));
00541
00542     // shift existing elements
00543     std::move_backward(begin() + index, end(), end() + count);
00544
00545     // copy new elements
00546     size_type i = index;
00547     for (const auto& v : ilist)
00548         data_[i++] = v;
00549
00550     size_ += count;
00551
00552     return begin() + index;
00553 }
00554
00555 template<typename T>
00556 template<std::ranges::input_range R>
00557 constexpr typename Vector<T>::it Vector<T>::insert_range(const_it pos, R&& rg){
00558     if (pos < begin() || pos > end())
00559         throw std::out_of_range("Vector::insert_range");
00560
00561     size_type index = pos - begin();
00562
00563     size_type original_index = index;
00564
00565     for (auto&& value : rg)
00566     {
00567         insert(begin() + index, value);
00568         ++index;
00569     }
00570
00571     return begin() + original_index;
00572 }
00573
00574 template<typename T>
00575 void Vector<T>::push_back(const T& value)
00576 {
00577     if (size_ == capacity_) reserve(capacity_ == 0 ? 1 : capacity_ * 2);
00578
00579     data_[size_] = value;
00580     size_++;
00581 }
00582
00583 template<typename T>
00584 void Vector<T>::push_back(T&& value) {
00585     if (size_ == capacity_) reserve(capacity_ == 0 ? 1 : capacity_ * 2);
00586
00587     data_[size_] = std::move(value);
00588     size_++;
00589 }
00590
00591 template<typename T>
00592 void Vector<T>::pop_back() {
00593     if (!empty()) size_--;
00594 }
00595

```

```

00596 template<typename T>
00597 void Vector<T>::resize(size_type count)
00598 {
00599     if (count == size_) return;
00600
00601     if (count > capacity_)
00602         reserve(count);
00603
00604     if (count > size_)
00605     {
00606         for (size_type i = size_; i < count; ++i)
00607             data_[i] = T{};
00608     }
00609
00610     size_ = count;
00611 }
00612
00613 template<typename T>
00614 void Vector<T>::resize(size_type count, const T& value)
00615 {
00616     if (count > capacity_)
00617         reserve(count);
00618
00619     if (count > size_)
00620     {
00621         for (size_type i = size_; i < count; ++i)
00622             data_[i] = value;
00623     }
00624
00625     size_ = count;
00626 }
00627
00628 template<typename T>
00629 typename Vector<T>::it Vector<T>::erase(const_it pos) {
00630     if (pos < begin() || pos >= end())
00631         throw std::out_of_range("Vector::erase");
00632
00633     size_type index = pos - begin();
00634
00635     std::move(begin() + index + 1, end(), begin() + index);
00636
00637     --size_;
00638
00639     return begin() + index;
00640 }
00641
00642 template<typename T>
00643 typename Vector<T>::it Vector<T>::erase(const_it first, const_it last)
00644 {
00645     if (first < begin() || first > end() ||
00646         last < begin() || last > end() ||
00647         first > last)
00648     {
00649         throw std::out_of_range("Vector::erase");
00650     }
00651
00652     size_type start = first - begin();
00653     size_type count = last - first;
00654
00655     std::move(begin() + start + count, end(), begin() + start);
00656
00657     size_ -= count;
00658
00659     return begin() + start;
00660 }
00661
00662 template<typename T>
00663 constexpr void Vector<T>::swap(Vector& other) noexcept {
00664     std::swap(data_, other.data_);
00665     std::swap(size_, other.size_);
00666     std::swap(capacity_, other.capacity_);
00667 }
00668
00669 //Testing
00670 template<typename Vec>
00671 auto measure_push_back(size_t count)
00672 {
00673     Vec v;
00674     size_t reallocations = 0;
00675     Timer timer;
00676
00677     for (size_t i = 0; i < count; ++i)
00678     {
00679         if (count == 100000000 && v.size() == v.capacity())
00680         {
00681             ++reallocations;
00682         }
00683     }

```

```
00683         v.push_back(i);
00684     }
00685
00686     volatile auto size = v.size(); //for compiler just in case it optimizes
00687     if(count == 100000000) cout << "\nis viso: " << reallocations << " atminties perskirstymu su
100000000 duomenu\n";
00688
00689     double res = timer.elapsed();
00690
00691     return res;
00692 }
```


Rodyklė

read

Studentas, [14](#)

Studentas, [13](#)

read, [14](#)

StudentuDuom - studentu duomenų apdorojimo aplikacija, [1](#)

Timer, [14](#)

Vector< T >, [14](#)

Zmogus, [15](#)